RELAZIONE Progetto C++ e Qt - Luglio 2025

- Marco De Meo
- 856627
- m.demeo@campus.unimib.it

C++

Il progetto è stato realizzato su una macchina Ubuntu 24.04.2 LTS, con compilatore g++ 13.3.0.

Struttura dati

Ho deciso di realizzare la sequenza di dati RLE come una linked list di coppie, ogni coppia rappresentata da una struct “pair” che al suo interno contiene un attributo per il valore templato V delle coppie, un attributo per il numero di ripetizioni successive di V, e un puntatore al nodo successivo.
Ho fatto questa scelta perché mi è sembrata la maniera più comoda di gestire la struttura dati richiesta, dandomi anche la possibilità di rimanere aderente all’approccio mostrato come esempio a lezione, nelle metodologie e nella forma.
Ho dotato la classe di un puntatore al primo e all’ultimo elemento della lista.
Quest’ultimo, sebbene possa essere forse ridondante, l’ho trovato comodo per gestire l’aggiunta in coda di dati alla classe, e nello specifico, nella mia implementazione, è importante anche per la gestione degli errori in questa operazione:
nel caso di errore in allocazione durante l’aggiunta in coda, mi aiuta a tenere traccia della fine della sequenza preesistente, passata a parametro, in modo da preservarne lo stato e poterla ritornare in modo coerente.
Mi sono reso conto successivamente che un'altra opzione sarebbe potuta essere quella di realizzare una sequenza temp: in caso di successo appenderla in coda alla sequenza preesistente sistemando i puntatori alla coda e al suo elemento successivo, e in caso di errore liberarmene; sono però soddisfatto dell’attuale gestione.
Ho inoltre dotato la classe di un attributo size, che rappresenta il numero di coppie presenti nella sequenza, e di un funtore binario per la valutazione dell’uguaglianza di due valori V.

Funzioni

1) Creazione istanza di classe RLE vuota

Questa funzionalità è realizzata dal costruttore di default, che inizializza la sequenza con size zero e puntatori a primo e ultimo elemento a nullptr.

2) Creazione istanza RLE a partire da coppia di Iteratori generici

Poiché una sequenza di dati codificati RLE può esistere solo come compressione di una sequenza di dati (o come istanza vuota), ho deciso di inserire la logica di creazione di una classe dentro alla funzione di codifica dei dati, e in essa anche la gestione delle diverse casistiche

3) Codifica di una sequenza all'interno di istanza RLE, eventuale aggiunta in coda

Come anticipato al punto precedente, questa funzione si occupa sia del riempimento di una sequenza vuota, gestendo la logica anche in fase di costruzione, sia dell'aggiunta di dati codificati in coda a una classe già esistente e già parzialmente riempita.
Per l'effettiva creazione di una nuova coppia e la sua aggiunta alla sequenza RLE, chiama la helper function append, che si occupa dunque della gestione degli attributi della sequenza.

Gli altri punti importanti della encode penso siano:

- il ciclo for: ho deciso di utilizzare un singolo loop, che a ogni iterazione sostanzialmente:
    A) raccoglie un match (ripetizione di un valore) e aumenta il conto (N)
    B) in caso di mismatch scarica valore V e ripetizioni contat N in un "pair"
  nota: questa logica rende necessaria l'aggiunta dell'ultimo pair fuori dal loop

- la gestione degli errori:
  come spiegato in introduzione, ho realizzato una funzione di deletion dei pair (utilizzata dal distruttore o altrimenti sempre wrappata in un blocco *try/catch*) che prende in ingresso un puntatore al pair da dove iniziare a cancellare, per distinguere tra errore durante la creazione o durante l'allocazione in coda.

4) # Complessivo dati codificati

Per il punto 4) della specifica di progetto, ovvero "Conoscere il numero complessivo di dati codificati", ero un po' indeciso. Ho scelto di ritornare semplicemente il numero di coppie contenute nella sequenza, per due motivi:

1. Per semplicità, avere una funzione che ritorna la size della mia struttura dati, tenendone traccia con un attributo membro, mi è sembrata la scelta più ovvia e coerente con la struttura della classe realizzata
2. Avrei potuto ritornare l'effettiva quantità di dati della sequenza passata in input, ma utilizzare un loop che somma tutte le N della sequenza RLE per conoscere la size della sequenza input, ho pensato fosse un dato di minore importanza e forse un po' una forzatura

5) Operatore di stream <<

Ho definito l'operatore di stream come funzione friend definita internamente alla classe RLE. Poiché il const_iterator proprietario della classe è limitato a percorrere la sequenza dati in essa contenuta, considerandone solo una rappresentazione in forma espansa, ed essendo l'operatore di stream obbligatoriamente globale, accedo alla classe tramite i suoi dati privati

(in questo caso un puntatore al primo *pair* contenuto nella sequenza), definendo la funzione friend della classe RLE.

6) Iteratore forward costante su sequenza espansa

Siccome questo iteratore deve ritornare la sequenza dati scompattata, dobbiamo fare in modo che ragioni con una rappresentazione di questo tipo, e si muova sulla sequenza con questa logica.
Per questo motivo, ho fatto due piccoli adattamenti alla classica interfaccia di un forward const_iterator:

1) la presenza di uno stato interno (‘*currCount’*), che viene associato al numero di ripetizioni N quando l’iteratore incontra un nuovo valore V
   Nota: ho dovuto inserire dei controlli per evitare di dereferenziare un nullptr cercando di assegnare il valore di N, in particolare nel costruttore entry point dell’iteratore, nella initialization list, ho usato per semplicità la sintassi:
   “*currCount(p ? p->_repCount : 0)*”,
   che è equivalente a:
   “*if(p != nullptr) currCount = p->_repCount; else p = 0;*”
   Non la abbiamo vista a lezione, però mi sono informato sul fatto che fosse un costrutto pre C++11.
2) la modifica dell’operatore di incremento:
   siccome è l’unico operatore che gestisce lo spostamento di un forward iterator sulla sequenza, è qui che bisogna inserire la logica di espansione.
   Dunque fa uso del contatore al punto precedente, in modo da spostarsi al valore successivo solo una volta esaurite le occorrenze di V.
   Nota:
   ho usato *if(--currCount == 0),* perché se avessi controllato *if(currCount > 0),* avrei passato una situazione del tipo <V, 0> all’iteratore, e avrei dovuto inserire un controllo anche nell’operatore di dereferenziamento, mentre così tutta la logica rimane dentro all’operatore di incremento.

NOTE

- Data la struttura della classe, in particolare la sua stretta dipendenza da sequenze passate in input, e la necessità di non mantenerne una copia interna, non ho ritenuto necessario lo sviluppo di altre funzionalità da ritenersi fondamentali per la specifica progettuale
- Nella funzione *swap* ho deciso di inserire uno scambio anche del funtore, qualora questo, in implementazione di oggetti custom complessi, potesse necessitare di stati interni di cui tenere traccia
- Ho deciso di definire delle typedef, sostanzialmente per i tipi di V ed N, perché ho pensato che potesse essere utile a un eventuale mantenimento del codice, e per uniformità
- Nell’esecuzione locale dei file, i test di passaggio a Valgrind hanno prodotto report positivo sull’utilizzo della memoria, allego screenshot della mia esecuzione:

```
==485282== HEAP SUMMARY:
==485282==     in use at exit: 0 bytes in 0 blocks
==485282==   total heap usage: 255 allocs, 255 frees, 80,360 bytes allocated
==485282==
==485282== All heap blocks were freed -- no leaks are possible
==485282==
==485282== For lists of detected and suppressed errors, rerun with: -s
==485282== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

QT

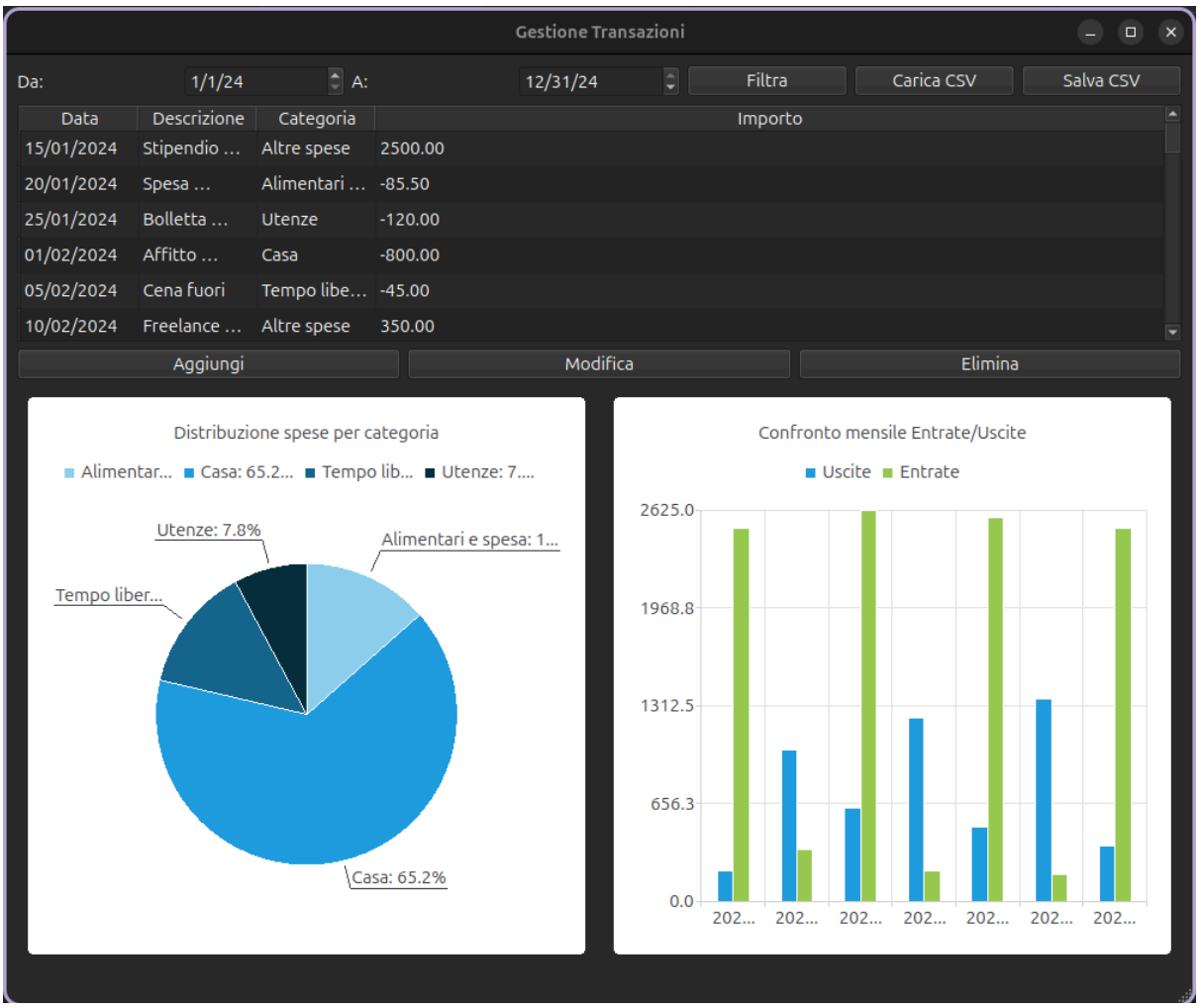
Il progetto è stato realizzato con Qt 6.8.1, su sistema operativo Ubuntu 24.04.2 LTS

Nota introduttiva: è molto più codice del C++, ed è pieno di costrutti di tante classi diverse con i loro metodi di utilities etc.
Per questo motivo cercherò di essere molto più sintetico e di parlare al volo della struttura, senza scendere nei dettagli.

MAIN WINDOW

Ho deciso di organizzare nella main window sia i widget dei grafici che quello della tabella per le transazioni, per avere un'unica finestra con tutto compatto e chiaro.
Per fare ciò ho utilizzato un *QVBoxLayout* come container principale per il layout (preso dal file UI), all'interno del quale in una sezione c'è il widget della tabella (già nel file UI), mentre nell'altra un container per entrambi i widget dei grafici, wrappato in un *QHboxLayout* che li dispone uno a fianco all'altro.
La main window contiene dei *button* che permettono di utilizzare le funzionalità principali richieste dalla specifica di progetto: I/O di CSV (*onLoadCSV, onSaveCSV*) e aggiunta, modifica, eliminazione o filtraggio di transazioni (*onAdd, onEdit, onDelete, onFilter*).
Questi triggerano dei dialog per selezionare i dettagli, ma utilizzano le funzioni definite internamente al modello di transazione per l'effettivo aggiornamento del modello, e si occupano anche del controllo sulle categorie possibili di transazione, in fase di aggiunta o modifica.
Ho limitato la logica di controllo delle categorie a questa sezione per semplicità, pensando a un utilizzo solamente da UI da parte dell'utente, nonostante questo lasci scoperta una vulnerabilità, dovuta alla possibilità di utilizzo delle funzioni pubbliche del modello di transazione da parte dell'utente, qualora non si limitasse all'uso tramite UI.
Le operazioni sui grafici sono invece affidate a un'unica complessa funzione *updateGraphs()*, che si occupa di aggiornare sia il grafico a torta che quello a barre, conseguentemente ai cambiamenti nel modello (aggiunta/rimozione/editing/filtro di transazioni).
Nota: ho pensato di trattare spese/ricavi come importi negativi/positivi delle transazioni, questo è importante in tutta la logica dell'applicazione, per esempio nel grafico a torta skippo i dati positivi perché voglio vedere la distribuzione delle spese.

Allego screenshot di come risulta la finestra in fase di test:



Modello di TRANSAZIONE

Ho realizzato il modello di transazione con un'estensione della classe *QAbstractTableModel* per avere una tabella 2D.
Ogni riga della tabella rappresenta una *struct* transazione, che contiene degli attributi per i campi della transazione, e una ridefinizione degli operatori di confronto di uguaglianza/disuguaglianza, necessarie per le operazioni di modifica e rimozione delle transazioni, per trovare un match con quelle selezionate e quelle nel vettore interno.
Le struct transazioni sono organizzate in due parametri privati QVector, uno per le transazioni totali presenti (*m_transactions*), e uno che raccoglie solo quelle filtrate (*m_filtered*), in modo da mostrare solo quelle una volta applicato il filtro al modello.
Gli ultimi due parametri privati sono le date che costituiscono gli estremi del filtro (*m_filterFrom,* data di inizio, e *m_filterTo*, data di fine).
La classe per il modello contiene le funzioni che gestiscono le effettive modifiche al modello per le operazioni di aggiunta, modifica, cancellazione e filtro di transazioni sul modello displayato (*addTransaction, editTransaction, removeTransactions, setDateFilter*), un paio di funzioni helper per le precedenti (*getTransaction,* chiamata anche nella main window, e *updateFilter*, che siccome è usata solo internamente ho messo privata) e alcuni override di funzioni virtuali di QAbstractItemModel (*rowCount, columnCount, data, headerData)*.